

Survival Analysis

Rui

r Sys.Date()

目录

1 Packages	1
2 Surv Model	2
3 Cox Regression	3
4 Surv Plots	3

1 Packages

- Survival Analysis
 - survival
 - survminer
- Plots of Survival Analysis
 - ggfortify
 - ggplot

```
{r message=FALSE,results='hide',warning=FALSE} library(survival) library(survminer)
library(ggplot2) library(ggfortify)
```

Dataset “lung” in package “survival” is of interest, which contains the information about survival in patients with advanced lung cancer from the North Central Cancer Treatment Group. Performance scores rate how well the patient can perform usual daily activities. Variables include:

- inst: Institution code
- time: Survival time in days
- status: censoring status 1=censored, 2=dead
- age: Age in years
- sex: Male=1 Female=2

- `ph.ecog`: ECOG performance score as rated by the physician. 0=asymptomatic, 1= symptomatic but completely ambulatory, 2= in bed <50% of the day, 3= in bed > 50% of the day but not bedbound, 4 = bedbound
- `ph.karno`: Karnofsky performance score (`bad=0-good=100`) rated by physician
- `pat.karno`: Karnofsky performance score as rated by patient
- `meal.cal`: Calories consumed at meals
- `wt.loss`: Weight loss in last six months (pounds)

```
{r} warning=FALSE,message=FALSE} attach(lung) # Survival Object
```

`survival::Surv()` creates a survival object, with censored data marked with “+” following up censoring time and death cases simply showing the death time. The result is usually used as a response variable in a model formula.

Two most important parameters:

- `time`: For right censored data, this is the follow up time. For interval data, the first argument is the starting time for the interval.
- `event`: The status indicator, normally 0=alive, 1=dead. Other choices are TRUE/FALSE (TRUE = death) or 1/2 (2=death). For interval censored data, the status indicator is 0=right censored, 1=event at time, 2=left censored, 3=interval censored. For multiple endpoint data the event variable will be a factor, whose first level is treated as censoring. Although unusual, the event indicator can be omitted, in which case all subjects are assumed to have an event.

```
{r} survobj = Surv(time, status==2)
```

2 Surv Model

For the overall survival analysis, just specify the null hypothesis with `Surv~1`, i.e., NO groups.

```
{r} sfit <- survfit(Surv(time, status==2)~1, data=lung)
```

The fitted survival model itself will tell you the results, with observations, events, median and 95% CI.

```
{r} sfit
```

If more details are needed, i.e., the instantaneous events, and estimates of survival, `std.err`, 95% CI, just `summary(sfit)`.

Moreover, if `time` is designated, then the `summary()` function will return the estimated results.

```
{r} summary(sfit,time = 200)
```

However, if group is of interest, specify it in the formula with `Surv~group`.

```
{r} sfit_sex = survfit(Surv(time,status==2)~strata(sex)) sfit_sex
```

From the model itself, we can get an overall picture of the differences between groups.

```
{r} summary(sfit_sex,time = 365)
```

It's ideal that the 95% CI of groups doesn't intersect, and a significant result can be obtained instantly. However, that's not the case, take a step further and use Cox regression to see exactly.

3 Cox Regression

```
{r} coxph(Surv(time, status==2)~sex)
```

where “`exp(coef)`” is the HR (hazard ratio), which measures the relative risk between two groups.

Theoretically, it's better only open to binary variables in Cox regression. However, for categorical variables with more than 2 levels, use “`as.factor(cat_var)`” with “`model = TRUE`” to generate corresponding dummy variables. For continuous variables, they'll be included into the regression model as usual, but the model will be plotted with expected value of continuous variables.

However, similar to `coxph()` which returns the differences through the view of regression, the simpler version of such comparison can be accomplished by “`survdif()`”, also in package “survival”. Both of the functions have the same formula.

From “`survdif()`”, we can get the intermediate products when computing the log rank, or the chi-square test. However, this wouldn't return the coefficient.

Jointly speaking, the combined results of two functions emphasize the fact that the log rank test (named Mantel-Haenszel test), is a chi-square test in essence.

```
{r} surv_diff = survdiff(Surv(time, status==2)~sex) surv_diff surv_diff$chisq
```

4 Surv Plots

Finally, we can visualize the survival analysis.

Note that even if analysed data has been attached, if the “`data`” isn't passed into `survfit()`, then an error will be raised.

The parameter “`risk.table`” provides you with the freedom to decide whether or not to add a risk table below the Kaplan-Meier curve(s). Meanwhile, various options are available!

```
{r warning=FALSE} ggsurvplot(sfit_sex,data = lung, conf.int=TRUE, pval=TRUE,
risk.table='abs_pct', legend.labs=c("Male", "Female"), legend.title="Sex",
title="Kaplan-Meier Curve for Lung Cancer Survival")
```

Moreover, the parameter “`fun`” is quite useful, which will define a transformation of the survival curve.

- “pct”: for survival probability in percentage, **by default**.
- “event”: plots cumulative events, $f(y) = 1 - y$.
- “cumhaz”: plots the cumulative hazard function, $f(y) = -\log(y)$.

```
{r} ggsurvplot(sfit_sex, data = lung,                conf.int=TRUE, pval=TRUE,                #  
risk.table='abs_pct',                legend.labs=c("Male", "Female"), legend.title="Sex",  
# title="Kaplan-Meier Curve for Lung Cancer Survival",                fun = 'event')  
  
{r} ggsurvplot(sfit_sex, data = lung,                conf.int=TRUE, pval=TRUE,                #  
risk.table='abs_pct',                legend.labs=c("Male", "Female"), legend.title="Sex",  
title="Kaplan-Meier Curve for Lung Cancer Survival",                fun = 'cumhaz')
```